

FIZZ DETECTIVE FINAL REPORT

ENPH 353 - Term 1 - December 17, 2023

Miti

Team 7 - Winux

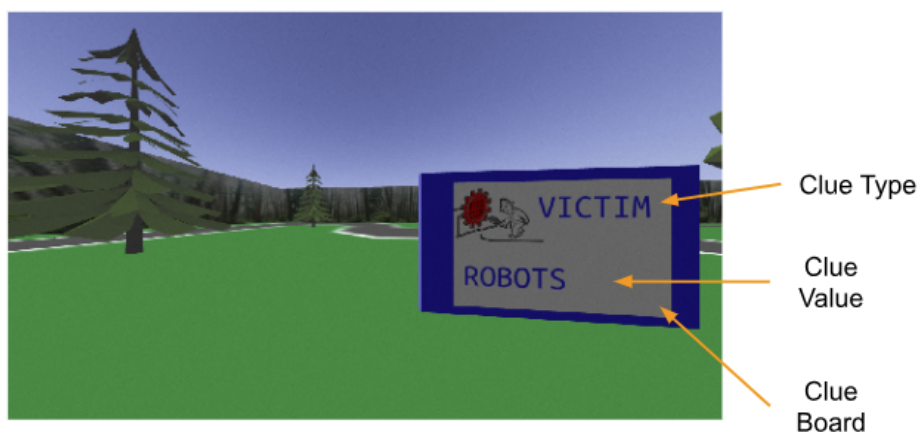
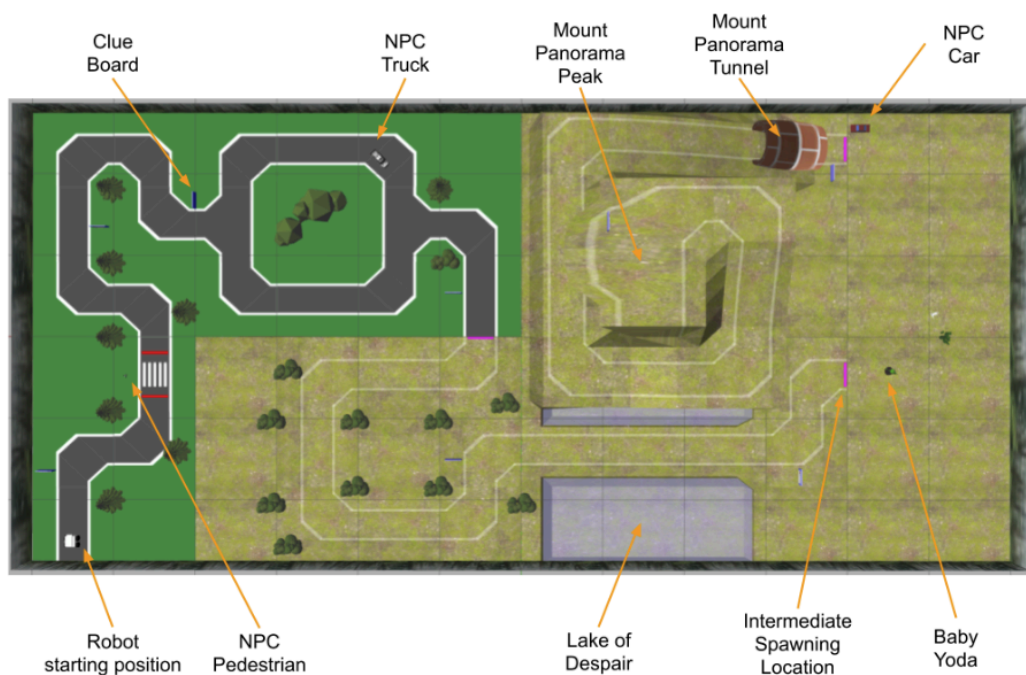
Itai Boss & Josh Heuchert

Table of Contents

Table of Contents.....	1
1. Introduction.....	2
1.1 Our Solution.....	3
1.2 Contributions.....	3
1.3 System Architecture.....	3
2. Driving.....	4
2.1 Data collection.....	4
2.2 Training.....	5
2.3 Obstacle avoidance.....	6
2.4 Controller.....	7
3. Clue Board Detection.....	8
3.1 Pre-Processing.....	8
3.2 Clue Detection Algorithm.....	9
3.3 Convolutional Neural Network.....	9
3.3.1 Data Collection.....	9
3.3.2 Training.....	10
4. Conclusion.....	11
4.1 Alternate Methods.....	11
References.....	12
Appendix.....	13
Imitation GUI.....	13
Top K losses.....	14
Driving NN summary.....	15
Pedestrian Mask.....	16
Yoda Mask.....	16
Sample Simulation Boards.....	17
Letter CNN summary.....	18
CNN Plots.....	18
CNN Confusion Matrix.....	19

1. Introduction

The purpose of this paper is to describe how our team, Winux, completed the ENPH 353 Fizz Detective Competition. The competition takes place in a simulated Gazebo environment (see below). The goal of the competition is to traverse the environment finding clues to solve the crime. An example clueboard can be seen below. A team gets points for each clue they submit and loses points for crashes, teleportations/respawns, or traffic violations. All of this is performed by an autonomous simulated agent that can access the camera, move, and submit clues. Our team developed an agent control system that performed better than any other team, collecting all points in the shortest amount of time to win the competition.



1.1 Our Solution

Our solution for this challenge utilized imitation learning to train driving models to accurately navigate the course, a convolutional neural network to read clue boards, and a state machine to control the agent. We divided the course into four main sections each with its own set of obstacles. First, we had to tackle the road section which has tight turns, clearly defined lines as well as a pedestrian to avoid and a roundabout with a moving NPC vehicle. The next section was the grass with less clearly defined lines and s-turns. Then we developed a system for navigating around NPC baby Yoda and returning back to the tunnel before the mountain. Lastly, all we needed to do to finish the run was make it through the tunnel and up the mountain, and submit the clues we had collected.

We trained four different models to tackle the driving within each section, allowing for a modularized system that was fit to specific data. This worked great, especially for sections like Yoda, where the agent was able to follow a predetermined path and did not rely on any road lines which would have been difficult to do with a PID implementation. As the agent drove around we would save images of the detected clueboard from a specified distance, and once it had reached the final clue on the top of the mountain we would submit all of the clues at once to finish the run. We also had backup features to submit all clues if the agent were to crash or go off course.

1.2 Contributions

To solve this challenge as a pair, the responsibilities were divided in the following way:

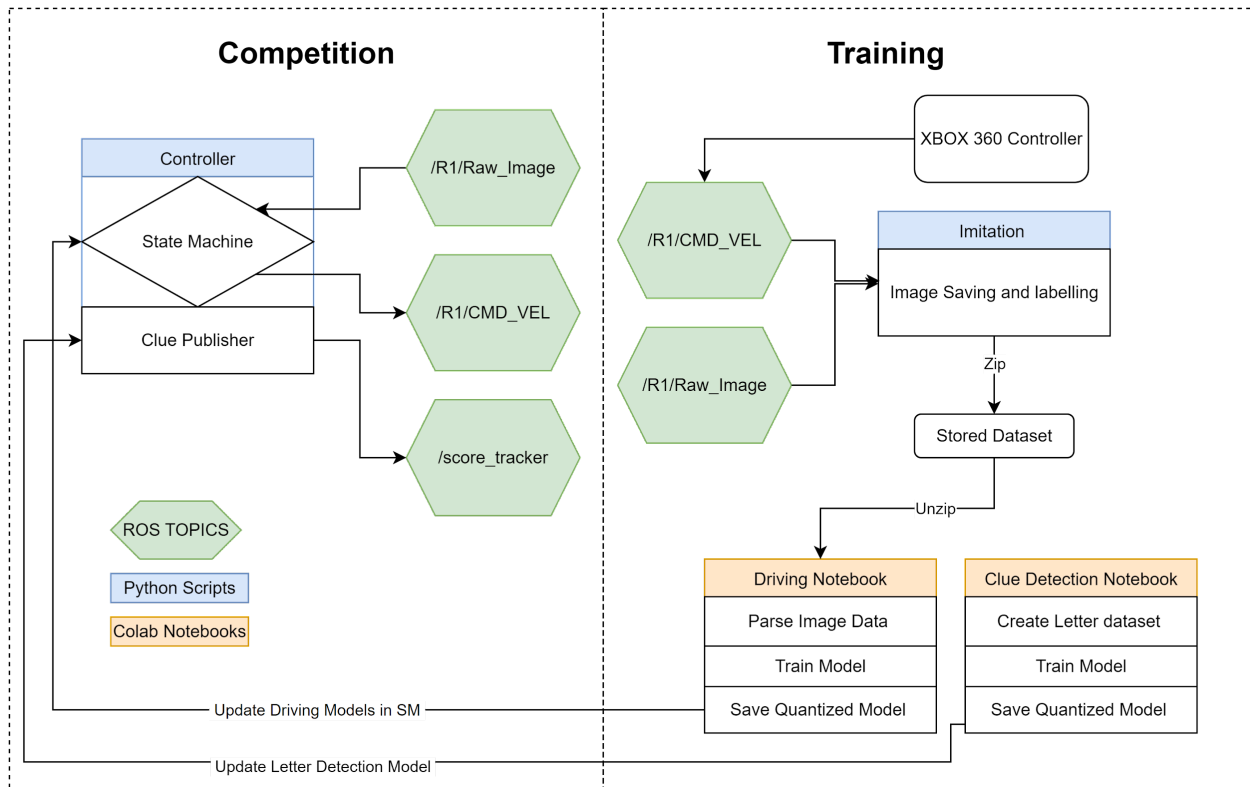
Itai's Contributions	Josh's Contributions
- Designed pipeline for collecting and labeling driving data with an Xbox 360 controller. - Designed and trained neural networks using imitation learning for navigating each section of the course. - Isolated the clue boards from the camera feed using homography and perspective transforms. - Constructed the state machine, event and callback architecture. - Developed solutions for pedestrian, vehicle and yoda detection.	- Gathered artificial as well as real simulation clue board data. - Trained the clue detection NN to reliably predict the clues. - Integrated the clue detector into the state machine. - Performed reliability testing. - Determined adequate measures to avoid the vehicle. - Developed a structured sequence to ensure reliable performance for competition.

1.3 System Architecture

To achieve a reliable driving controller and clue detection system we worked on two different subsystems. We developed one for testing the state machine and image processing that would need to be done by the controller in competition, and the other for data generation. The data generation pipeline consisted of a python script designed to save and label human driving images from data in the ros camera and velocity topics, which would be passed to a colab notebook to train driving models. Within this system there was

also a second colab notebook for generating modified and labeled letter images and training the clue detection model.

The output of these pipelines were quantized tensorflow-lite models that could be directly implemented in the competition controller. Having these subsystems allowed for testing, training and debugging to be done in parallel. We were able to create many iterations of each model and with the feedback we were seeing in the competition subsystem, quickly adjust the models to perform more accurately.



2. Driving

To navigate the course, we developed a robust pipeline for data collection, network training and model implementation. The models take grayscale compressed images from the camera feed and output a steering angle for the vehicle. We used four separate driving models for each section of the map.

2.1 Driving Neural Network

2.1.1 Data collection

To train reliable models we needed to collect thousands of images with labeled driving inputs. We developed a GUI and script with a text input for a file path. This would write the saved images and label them with the corresponding command velocity topic parameters for steering angle and throttle values. The GUI allowed us to drive solely based on the camera feed, like the car would, as well as start and stop recording easily to drive back and re-record various sections. [see Imitation GUI]

To drive effectively and quickly around the course we opted to train off of human input using a joystick controller. This allowed for a range of possible steering inputs rather than the three possible states when driving with keyboard-teleop and had a great impact on our ability to take smooth corners and not have to stop for sharp turns. Using a wired Xbox360 controller we were able to use the existing ROS packages `joy` and `teleop_twist_joy` to convert the controller input to `/R1/cmd_vel` messages that would move the robot. We then modified the teleop launch file to map the controller buttons and set the angular and linear speeds. Using this we could rapidly experiment with collecting data at various speeds and optimize our training for high-speed laps.

Over the course of training the models, we collected ~10,000 images per region of the course which we saved to dedicated folders such as `/Road/` with subfolders for subsections like `/Turn1` and iterations of that section. This gave us a great database to pull from when training and allowed us to modify our dataset to follow specific paths and fix certain turns. Since the images used for training were human driving data there was variation between driving lines so having a lot of data greatly improved our model's reliability.

2.1.2 Training

Once we had a large dataset to train from we developed a colab notebook to import the data, create training and validation sets, train our model, and save the model. We also implemented debugging tools for verifying model accuracy. Since we were doing this in colab all of the images would need to be uploaded through Google Drive. We found an effective method to easily move all of these images was to zip them, import the zip to drive, and then unzip to get the images in the notebook.

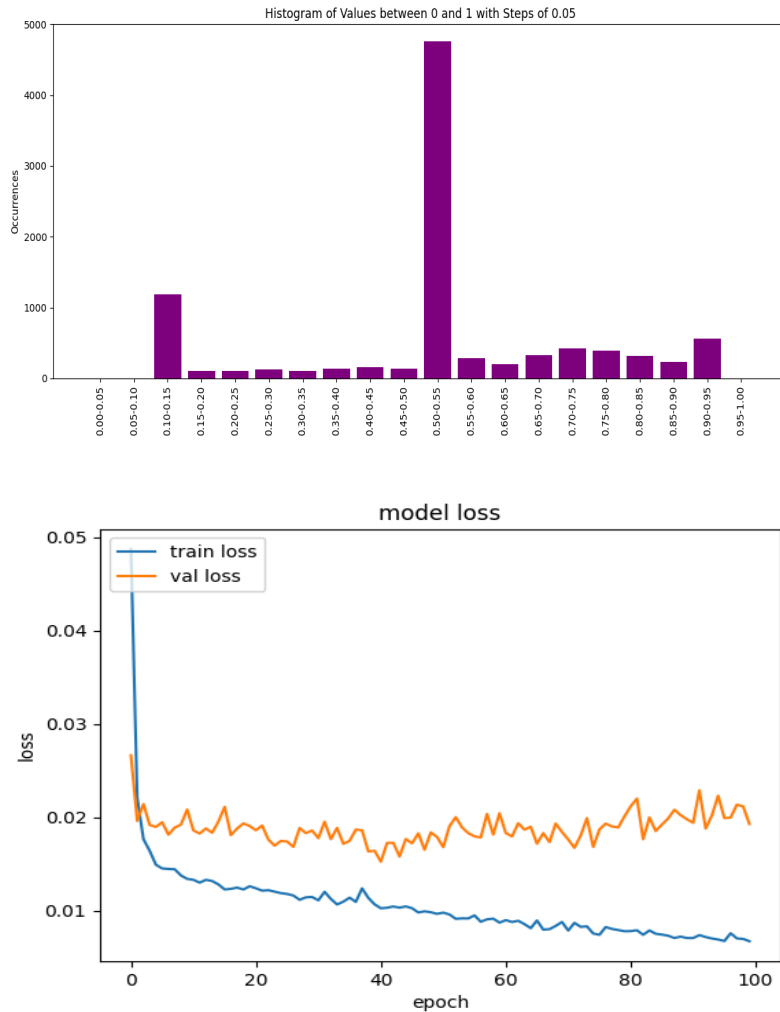
Once we had unzipped the folder for the region we wanted to train the model, we could iterate through each image and parse its label to create two arrays, one with the images to feed into the network, the other being the steering label of the corresponding image. During this process, we would set aside one full run as a validation set which was used to determine how the model fit.

The neural network architecture implemented was inspired by the paper “End to End Learning for Self-Driving Cars [1].” It features an input layer followed by five 2D convolution layers, a flattening layer and then four fully connected layers before its output. The input layer dimensions were `[90,160,1]` to match 1/8 scaled down and grey scaled driving images of `[720,1280,3]`.

To reduce the overall number of parameters to not overuse the limited resources available, we used the scaled-down images, increased the stride size on the 2D convolutions and lowered the overall number of nodes in the fully connected layers. In total, the network had ~700,000 tunable parameters [see Driving NN summary]. When training we used a learning rate of $1e-4$ as well as using the Adam optimizer. We found that with a dataset of around 10,000 inputs or `[10000,90,160,1]` and a batch size of 16, our models were able to fit after 80 epochs.

It was important for us to visualize how many images of straight, left, and right turns we were training from, as we did not want the model to fit to drive straight. We created a simple function to plot a histogram of the training and validation labels. Although the majority of our training data was on

straightaways, for the regions with many turns we supplemented the dataset with extra turn data to achieve a 2:1 ratio of straightaways to turns.



Some other tools used to verify our model was training properly were to visualize the losses for our training and validation data, as well as looking at the images that had the largest losses. Looking at the loss plot it looks as if our system is overfitting from the start. At first, we were alarmed, but when testing the models they worked quite well. Once we looked at the images of the top losses we realized that in fact the models were well fit, and it was the human driving inputs that were wrong. This is an artifact of how we made quick correction inputs with the joystick that were not reflective of the actual desired turning. This phenomenon can be seen in [Top K losses].

To further improve the reliability of these models when implemented in the driving controller it was critical to decrease the forward pass time to avoid missing frames or driving from delayed inputs. To achieve this we moved away from saving and loading

the models with keras and instead opted for quantized tensorflow-lite models [2]. This optimized network performs integer operations instead of floating point operations, greatly decreasing computation time without a large sacrifice in output accuracy.

2.2 Obstacle avoidance

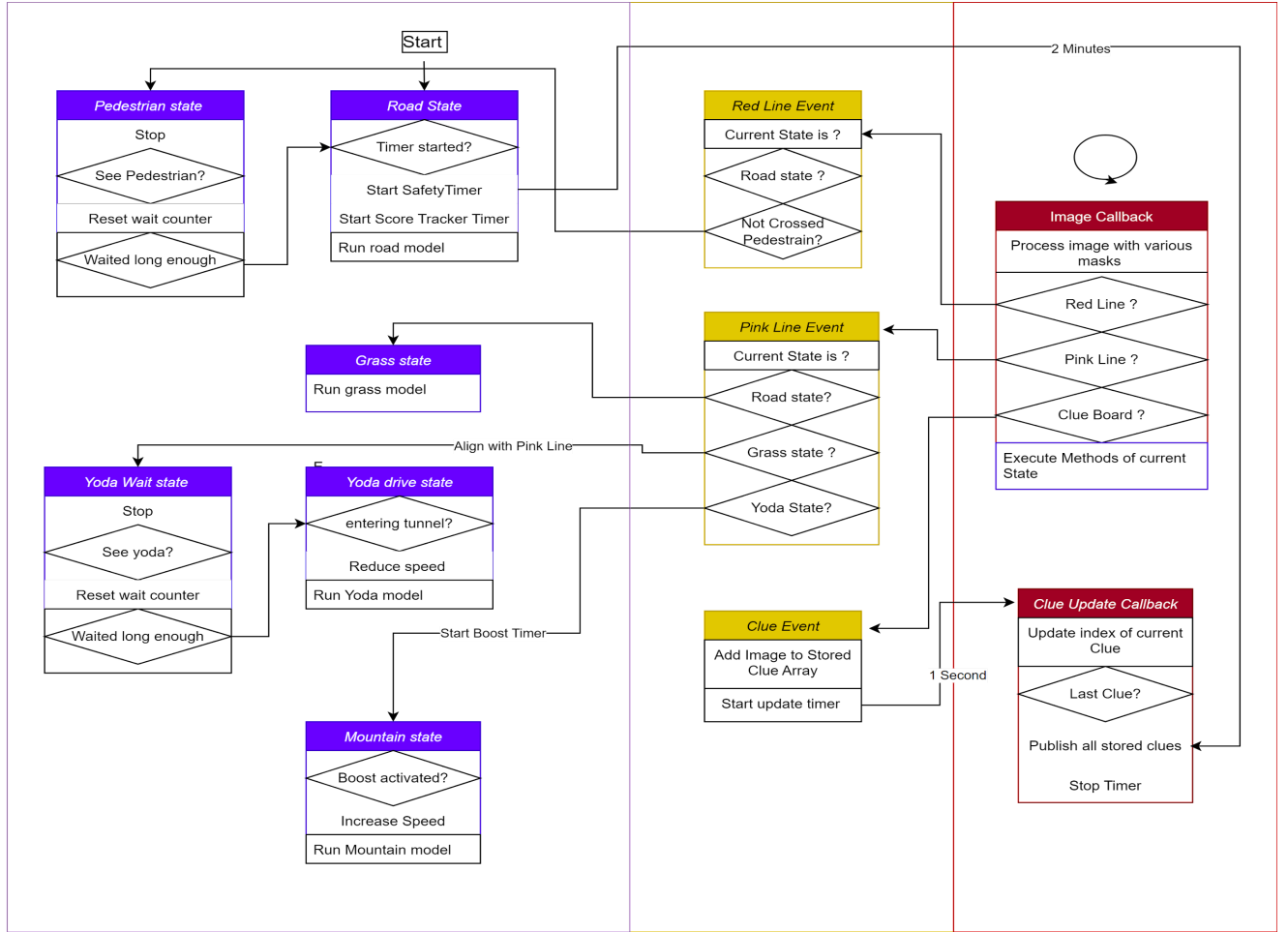
For object detection we opted for a classical image processing approach. For the pedestrian and Yoda, the approach was to stop at an indicator that we had arrived at the crossing (red and pink lines). Once we had stopped we were able to use openCV to create masks and subtract frames to detect moving objects in the field of view. This only worked when the robot was not moving, which is what forced us to stop at the intersection indicators. [see Pedestrian mask & Yoda mask]

For the pedestrian, a background subtraction mask was not enough since the vehicle at the roundabout was also driving in the background. For this section, we created a mask based on the pants color of the pedestrian and then used background subtraction to track whether the pants were moving [Pedestrian mask]. For Yoda we could not just track its movement since he would take long pauses in the middle of our desired path, so instead we created a color mask for Yoda [Yoda mask]. For both sections, if the mask used had a contour that was greater than a certain size, it was deemed unsafe to cross. We would wait at the intersection until a predetermined number of frames where no object was detected. Although we developed a similar system for detecting the vehicle, we did not end up implementing it in our final controller as we found that it greatly slowed us down. We realized that we could time our start to match the vehicle and never end up on the same side of the roundabout.

To navigate around Yoda we trained the model to follow a predetermined U-shaped path around the cactus and back to the tunnel. As a result, we had to wait for Yoda twice, but it allowed us to be aligned well for entering the tunnel.

2.3 Controller

To navigate through the course effectively we designed a finite-state machine system that implemented events, timers and callbacks. The state machine executes a routine to control the robot depending on its current state. For our implementation, we had six states, one for each section of the course (road, grass, no markings, and mountain), as well as one for each obstacle encountered (pedestrian, Yoda). Callbacks are the methods executed as a result of various ROS topics and timers. In our case the main callbacks used were for each time the robot published an image to the `/R1/raw_image` topic and for timers that were set in other subroutines. Events occur when a specific ‘event’ is determined by methods within the camera callback, these are analogous to interrupts. This approach allowed for modularity, ease of debugging, ease of modification and reuse of code. Below is an overview of the States, Events and Callbacks used in our controller.



Since we were using a NN for driving we would be passing images through the model on every frame. To ensure the speed of our callback loop we made each state and event as short as possible, as well as quantizing the driving models to ensure that we were not executing routines slower than the camera images were being published. Other solutions to this problem that we implemented were to run the simulation with a lower real-time factor, as well as having the laptop charging, disconnected from wifi and no other background tasks to maximize the portion of the CPU that could be dedicated to running the simulation.

3. Clue Board Detection

3.1 Pre-Processing

To detect the clues, we used classical computer vision (CV) techniques along with a convolutional neural network (CNN). The first step was to determine whether there was a clueboard in the agent's vision. This was done by isolating the non-blue region within a blue bounding area. This non-blue region is the clueboard which is then perspective transformed, to be a flat image with our desired dimensions. This was done by approximating it as a polygon and using the corners to transform the image. To improve the

image quality, we converted to grayscale and then applied CLAHE (contrast limited adaptive histogram equalization). CLAHE improves the contrast of images, making details more visible. These clue boards were then stored in arrays for future processing. [See Sample Simulation Plates].

3.2 Clue Detection Algorithm

To avoid disruptions in processing driving inputs, the clues were only computed and submitted twice throughout the course. The first analysis occurred when the system reached the Yoda checkpoint and the second was at the end of the course. Additionally, there was a failsafe mechanism to submit the clues after 120 seconds of simulation time elapsed. This ensured that points would be scored, even in cases where driving encountered challenges along the course.

When computing and submitting the clues, some additional measures were in place to ensure appropriate submissions. For each board, only the 5 most recent plates were computed and the average of the valid clues were submitted. The clue was 'valid' if its key (the clue type) was in the set of all possible clue types. For each image, the key and value were determined by splitting the board into letters, taking advantage of the fact the letters were always in the same position on the board. Then each letter was considered an empty space or predicted through the neural net.

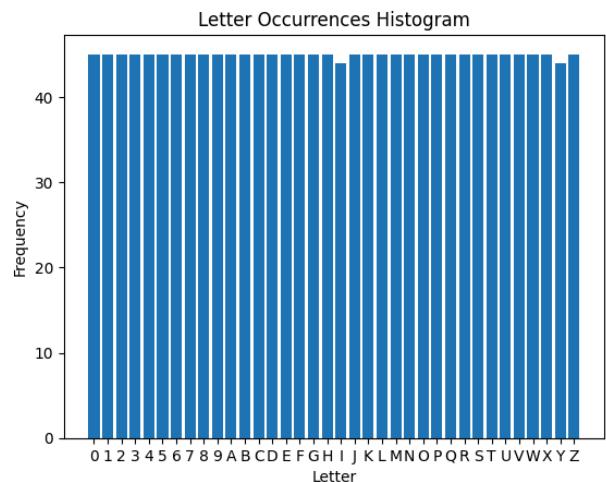
3.3 Convolutional Neural Network

The convolutional neural network that predicted the letters was made in a colab notebook using the keras library. The model was sequential and the layers and parameter summary can be seen in [Letter NN Summary]. Similar to the driving neural networks, the network was imported into the controller as a quantized tensorflow-lite model for efficiency.

3.3.1 Data Collection

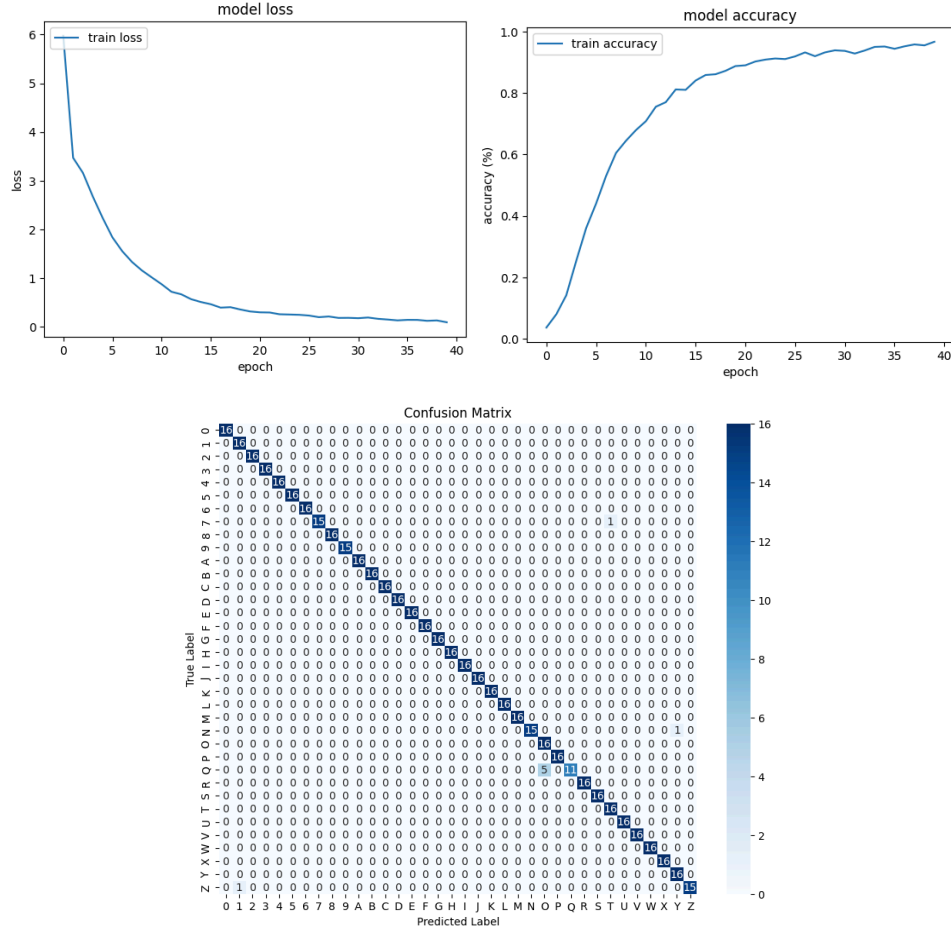
The model was trained on letter images generated in colab. To make the training robust, we utilized the keras image data generator function to assist in randomly disturbing the images. This was done on clean images as well as varying levels of blurred images. The resulting size of the dataset was about 45 images per letter or 1618 in total.

As seen in the histogram of training data, there were some letters with slightly less and this is a result of the random generator creating duplicates and overwriting the previous file. This was not a concern as the histogram still looked very smooth.



3.3.2 Training

The model was trained with a learning rate of $1e-4$ and a loss of categorical cross-entropy. The model was trained for 40 epochs with a batch size of 30. Training the model with no validation split also lead to the best results. The plots of the loss and accuracy after training can be seen below. Further testing was done on some validation data which had more blurred letters. The resulting confusion matrix is shown. This confusion matrix was representative of how the CNN performed on license plates in the simulation on real clue boards. As we can see, the network's only mistake is that some O's are predicted as Q's.



By analysing the probabilities of the output, it was discovered that the model could predict Q's correctly with much higher accuracy (did not mistake a Q as O with high probability). So, an additional algorithm was added that if it predicted a Q but its probability to be an O was $> 10^{-4}$ then change the prediction to an O instead. This simple algorithm significantly improved this error. [3]

4. Conclusion

In the competition, our agent was able to successfully secure all 57 points by submitting the correct clues with no driving errors. Our agent also completed the whole course in the shortest amount of time leading to our team winning the competition. Our agent successfully completed the course in 73 seconds of simulation time while the next fastest complete run was in 89 seconds. This was also one of our slower runs as during the reliability testing the night before the competition, we were averaging lap times in the mid-60s with our fastest recorded lap being 59 seconds. This slower run was due to unfortunate Yoda timing leading us to wait longer than usual. Despite winning the competition, our team was proud that we were able to successfully take on the challenge of imitation learning to complete the course and that we developed an architecture that was able to consistently get full points.

4.1 Alternate Methods

Some of our work was not included in the competition strategy that is worth expanding on. This section briefly talks about each of those methods.

We had trained models for throttle inputs, allowing the neural network to control the speed. However, we deemed this to be an unnecessary complication of the problem as the data we were collecting was of driving at a constant velocity. This approach was tested as a possible solution for object detection and showed some promise but was abandoned for the sake of time. Given more time, we would have found it interesting to develop a network that could do both steering and obstacle avoidance.

We also developed an implementable NPC vehicle detection with the architecture in place to implement it. It was very similar in structure and performance to the pedestrian and Yoda detection implementations. However, we decided against utilizing this in our competition to save time as we found if we timed our start correctly, we would not have to wait for the vehicle but were never at risk of encountering it.

Additionally, there were many convolutional neural networks trained purely on simulation data. This was expected to perform better as this is more realistic for the competition, however, these networks always introduced more errors that were unpredictable and impossible to correct by analyzing the probabilities. Given more time, we would have liked to analyze what was going wrong and train the CNN using simulated instead of artificially generated data.

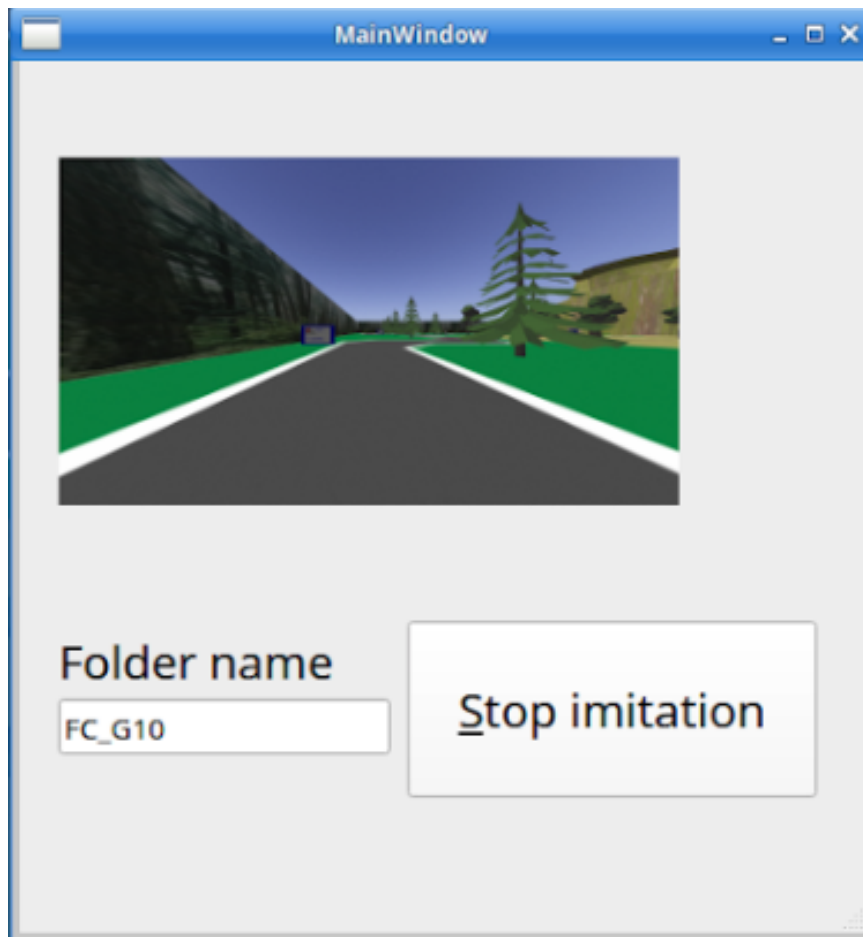
References

- [1] End to End Learning for Self-Driving Cars Url: <https://arxiv.org/abs/1604.07316>
Bojarski, M., Del Testa, D., Dworakowski, D., Firner, B., Flepp, B., Goyal, P., Jackel, L. D., Monfort, M., Muller, U., Zhang, J., et al. (2016). End to end learning for self-driving cars. arXiv preprint arXiv:1604.07316.
- [2] Discussion with Luca Cappellano on optimizing neural net forward pass with integer quantization. Before talking to Luca I was noticing issues with the reaction time of the network, and as result this recommendation was essential for our success, and led to us to quantizing all of our networks.
- [3] Discussion with Sasan Ghasaei on prediction confidence as letter prediction output of Neural Network. This led to the idea of fixing the O's and Q's issue by setting a prediction confidence threshold.

Appendix

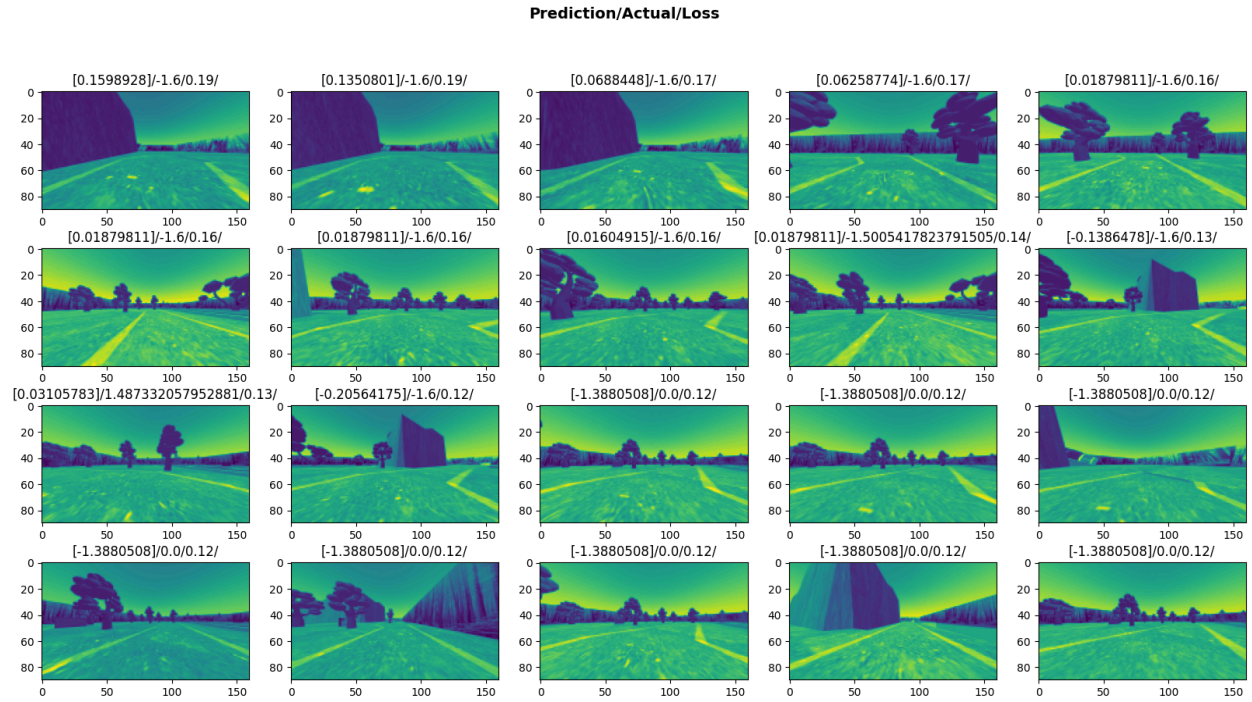
Imitation GUI

Image of the imitation GUI used for driving data collection.



Top K losses

Showing the images in validation with the top K losses. Showing the prediction/actual/loss for verification. This showed us that even in the areas of high losses, it was predicting reliably.



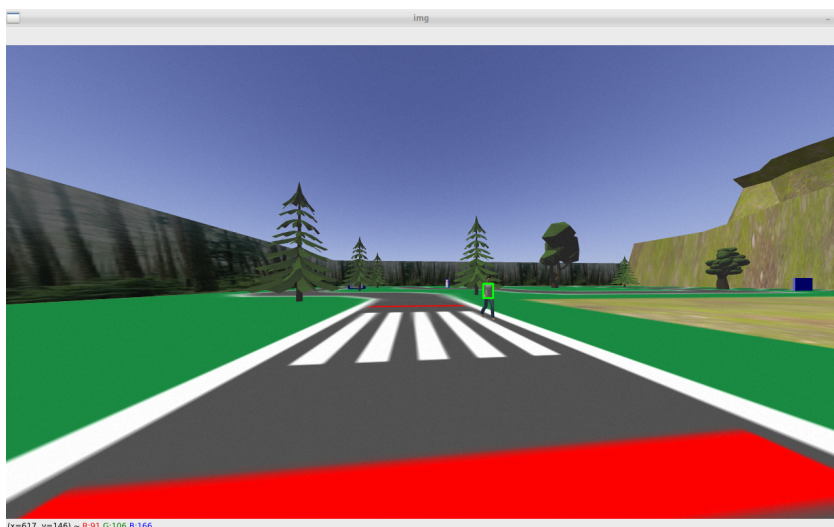
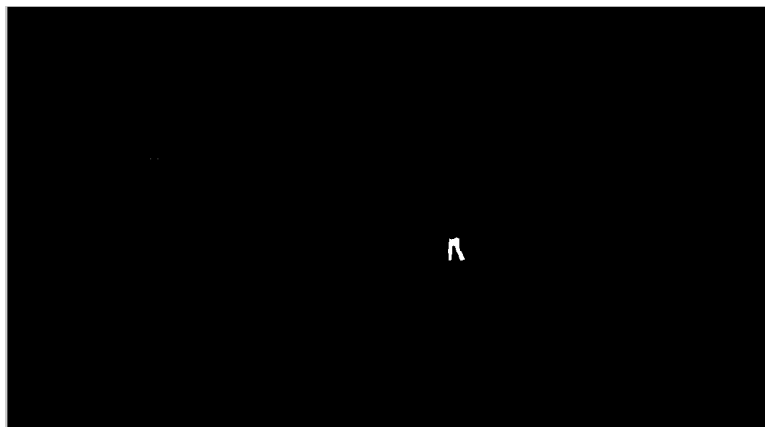
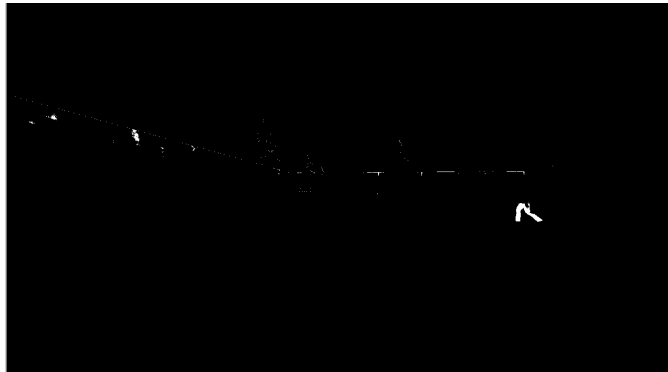
Driving NN summary

Summary of the driving neural network, displaying its inputs, layers, and parameters.

Model: "sequential"		
Layer (type)	Output Shape	Param #
lambda (Lambda)	(None, 90, 160, 1)	0
conv2d (Conv2D)	(None, 43, 78, 24)	624
conv2d_1 (Conv2D)	(None, 20, 37, 36)	21636
conv2d_2 (Conv2D)	(None, 8, 17, 48)	43248
conv2d_3 (Conv2D)	(None, 3, 8, 64)	27712
conv2d_4 (Conv2D)	(None, 1, 6, 64)	36928
flatten (Flatten)	(None, 384)	0
dense (Dense)	(None, 1164)	448140
dropout (Dropout)	(None, 1164)	0
dense_1 (Dense)	(None, 100)	116500
dropout_1 (Dropout)	(None, 100)	0
dense_2 (Dense)	(None, 50)	5050
dropout_2 (Dropout)	(None, 50)	0
dense_3 (Dense)	(None, 10)	510
dense_4 (Dense)	(None, 1)	11
Total params: 700359 (2.67 MB)		
Trainable params: 700359 (2.67 MB)		
Non-trainable params: 0 (0.00 Byte)		

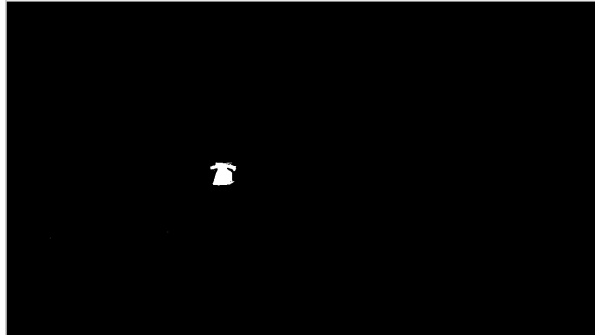
Pedestrian Mask

Images of the pedestrian mask and the pedestrian mask after background subtraction for pedestrian detection. The final image is the detected pedestrian in the original frame. (Photos taken at different instances)



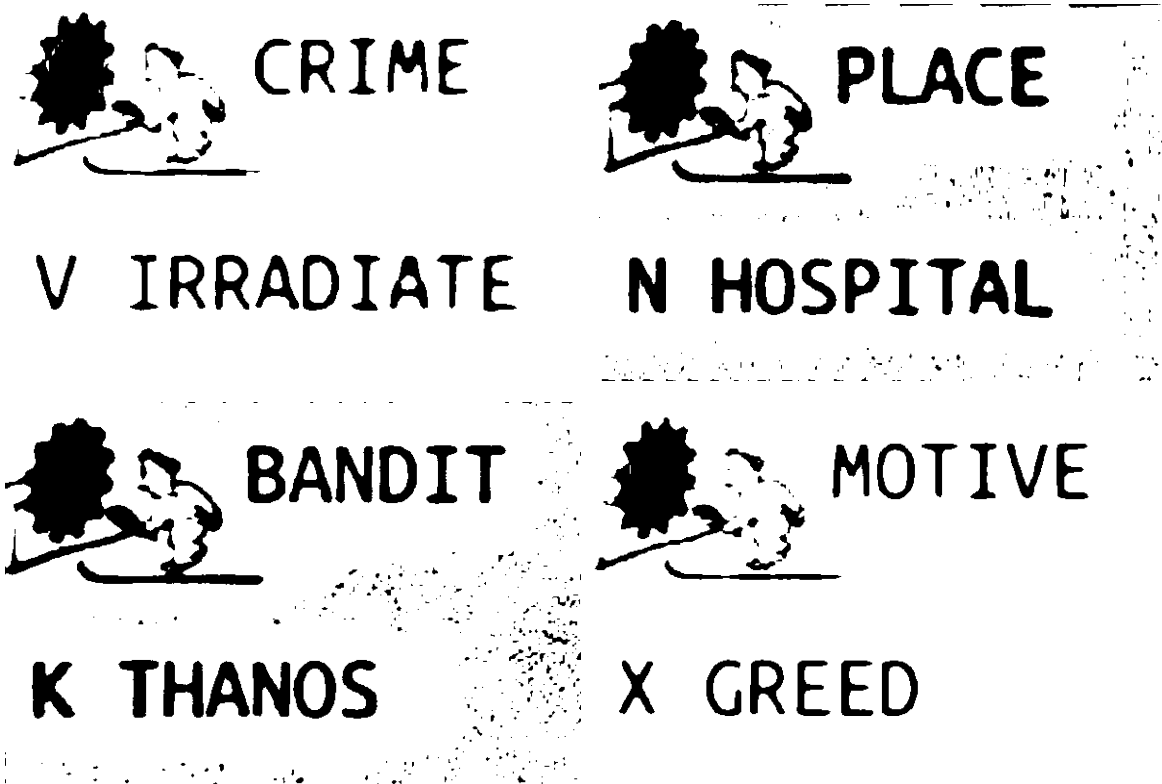
Yoda Mask

Image of the baby Yoda mask.



Sample Simulation Boards

Some clue boards that were grabbed from the simulation with our driving networks. These show some of the better and worse quality pictures.



Letter CNN summary

Summary of the letter convolutional neural network, displaying its inputs, layers, and parameters.

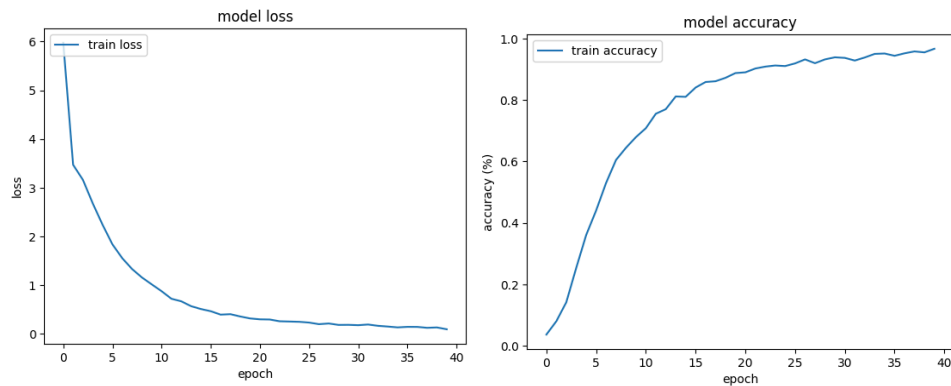
```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 73, 43, 32)	320
max_pooling2d (MaxPooling2D)	(None, 36, 21, 32)	0
conv2d_1 (Conv2D)	(None, 34, 19, 64)	18496
max_pooling2d_1 (MaxPooling2D)	(None, 17, 9, 64)	0
conv2d_2 (Conv2D)	(None, 15, 7, 128)	73856
max_pooling2d_2 (MaxPooling2D)	(None, 7, 3, 128)	0
conv2d_3 (Conv2D)	(None, 5, 1, 128)	147584
max_pooling2d_3 (MaxPooling2D)	(None, 3, 1, 128)	0
flatten (Flatten)	(None, 384)	0
dropout (Dropout)	(None, 384)	0
dense (Dense)	(None, 512)	197120
dense_1 (Dense)	(None, 36)	18468

```
=====  
Total params: 455844 (1.74 MB)  
Trainable params: 455844 (1.74 MB)  
Non-trainable params: 0 (0.00 Byte)
```

CNN Plots

Plots of the letter CNN's loss and accuracy over the training.



The confusion matrix of the letter CNN after validation testing.

